

Lab 2

[Start Assignment](#)

- Due May 1 by 11:59pm
- Points 10
- Submitting a file upload
- Available after Mar 27 at 12pm

ECE 446:04/565 (Spring 2025) Lab2: Introduction to Intel SGX

In this lab assignment, we will install Intel SGX SDK and run the SGX program in simulation mode on Ubuntu 20.04/22.04. The bonus part includes writing a digital wallet with SGX and running the SGX program in hardware mode.

Required Operating System: Ubuntu 20.04/22.04

Code needed for this lab: [lab_sgx.zip](#) 

(https://drive.google.com/file/d/1i4JMdZZK7jm51hKbJhiBi9N78U1_CPku/view?usp=sharing)

If you have any questions about the lab, please post them on the course Piazza site. Our graduate student helper Ke Xia (ke.xia@rutgers.edu (<mailto:ke.xia@rutgers.edu>)), who has kindly helped design this lab project, can answer your questions on Piazza.

1. Install SGX in simulation mode

Step 1: Build Intel SGX SDK installer

1. Open the terminal, and use the following command to install the required tools to build the Intel SGX SDK:

```
$ sudo apt-get install build-essential ocaml ocamlbuild automake autoconf libtool wget python-is-python3 libssl-dev git cmake perl
```

2. Create the folder "sgx_lab" and enter:

```
$ mkdir sgx_lab && cd sgx_lab
```

3. In the "sgx_lab" folder, download the Intel SGX SDK source code and prepare the submodules and prebuilt binaries:

```
$ git clone https://github.com/intel/linux-sgx.git
```

```
$ cd linux-sgx && make preparation
```

4. [Updated 3/30/2024] If you are using Ubuntu 22.04, please skip this step. If you are using

 Ubuntu 20.04, copy the mitigation tools corresponding to ubuntu distribution from external/toolset/ubuntu20.04 to /usr/local/bin and make sure they have execute permission:

```
$ sudo cp external/toolset/ubuntu20.04/* /usr/local/bin
```

Then you can test the location of the tools:

```
$ which ar as ld objcopy objdump ranlib
```

5. Build the Intel SGX SDK installer. It will take a while to complete the building process:

```
$ make sdk_install_pkg
```

When the installer is built, you can find the installer (.bin) in the directory "linux/installer/bin". The file name should be sgx_linux_x64_sdk_\${version}.bin

Step 2: Install Intel SGX SDK installer

1. Use the following command to install the required tool to use Intel SGX SDK:

```
$ sudo apt-get install build-essential python-is-python3
```

2. Invoke the installer as follows. You should use the complete installer name that you found in the previous step. "--prefix ../../../../" specifies the installation path as "sgx_lab/".

```
$ cd linux/installer/bin  
$ ./sgx_linux_x64_sdk_${version}.bin --prefix ../../../../
```

After the installation, you will see a message that is similar to the following:

```
"Installation is successful! The SDK package can be found in ${sgx-sdk-install-path}  
Please set the environment variables with below command:  
source ${sgx-sdk-install-path}/environment"
```

Then please set the environment variables with the command shown in the terminal. The command would be similar to the following, where the \${sgx-sdk-install-path} would change accordingly:

```
$ source ${sgx-sdk-install-path}/environment
```

Note: Every time you open a new terminal, you need to set up the environment variables.

2. Run the SGX "hello world" example in simulation mode

Submission: Please save all the terminal output to the file "Lab1_HelloEnclave.txt" and submit.

You can find the SGX "hello world" example in the folder "Lab1_HelloEnclave". The program mainly consists of two parts:

- The folder "App" includes the code for the untrusted application;
- The folder "Enclave" includes the code for the enclave.

Also, we have the file "Enclave.edl" in the folder "Enclave" to specify the functions that can be called into (ECALLs) and out of (OCALLs) the enclave.

If you open a new terminal, please set up the environment variables. Then you can start to compile the SGX program. Please run the following command to compile the program in simulation mode:

```
$ cd Lab1_HelloEnclave
$ make SGX_MODE=SIM
```

After the compilation, you will see the following message:

```
The project has been built in debug simulation mode.
make[1]: Leaving directory '${sgx-lab-path}/sgx_lab/Lab1_HelloEnclave'
```

Then you can run the program:

```
$ ./app
```

The following message indicates that the program runs successfully:

```
Hello world
```

If you encounter an error during the program compilation, please refer to the "troubleshooting" section.

3. Implement a random number generator with SGX

Submission: Please save all the terminal output to the file "Lab2_WriteEnclave.txt " and submit.

Please open the folder "Lab2_WriteEnclave". The code in the C++ file "rng.cpp" is a random number generator. Here, we declare the function "int gen_random_number()" to generate a random number:

```
#include <iostream>

using namespace std;

int gen_random_number() {
    srand((unsigned) time(NULL));
```

```
int random = rand();  
return random;  
}  
  
int main() {  
    int random_number;  
    random_number = gen_random_number();  
    printf("Random Number: %d\n", random_number);  
    return 0;  
}
```

We can compile and run this C++ program:

```
$ g++ -o rng ./rng.cpp  
$ ./rng
```

The output will be similar to the following, although the number will change:

```
Random Number: 2140173612
```

In cryptography, random numbers play a very important role. Assume we need an encryption key to protect our secrets. This encryption key should be generated randomly in such a way that nobody else can generate or possess the same key. If the random number generator is compromised or predictable, the encryption key is no longer secure. In this lab, we will use the SGX enclave to generate a random number.

You can find the template SGX program "SampleEnclave" in the folder "Lab2_WriteEnclave". To write an SGX-secured random number generator, first we need to define the part that we want to put into the enclave, i.e., define the ECALL "int ecall_gen_random_number()" in the Enclave Definition Language (EDL) File:

"Lab2_WriteEnclave/SampleEnclave/Enclave/Enclave.edl":

```
enclave {  
    from "sgx_tstdc.edl" import *;  
    trusted {
```

```

    /* Define a secure ECALL to
generate a random number */
    public int
ecall_gen_random_number();

};

untrusted {

    /* You can define OCALLs here if
needed */

};

};

```

In the next step, we implement the ECALL "int ecall_gen_random_number()" in the enclave file:

"Lab2_WriteEnclave/SampleEnclave/Enclave/Enclave.cpp":

```

#include "sgx_trts.h"

#include "Enclave_t.h"

/** Securely generate a random number inside the enclave. */
int ecall_gen_random_number() {

    unsigned int random_number = 0;

    sgx_status_t status = sgx_read_rand((unsigned
char*)&random_number, sizeof(random_number));

    if(status != SGX_SUCCESS) {

        return -1;

    }

    return (int)random_number;

}

```

The function "sgx_read_rand()" is used to generate a random number securely within the enclave. This function is part of the SGX trusted libraries and provides a more secure way to generate randomness compared to standard C library functions.

Then we can call the ECALL from the Untrusted Application in the App.cpp file. Here is the code for the main function in "Lab2_WriteEnclave/SampleEnclave/App/App.cpp":

```
/**
 * Main application.
 */
int SGX_CDECL main() {
    int random_number;
    initialize_enclave();
    sgx_status_t state = ecall_gen_random_number(global_eid, &random_number);
    if (state != SGX_SUCCESS) {
        printf("Can not run enclave call, error-code: %d\n", state);
        return -1;
    } else {
        printf("Random Number: %d\n", random_number);
    }
    sgx_destroy_enclave(global_eid);
    return 0;
}
```

After completing the SGX program, you can compile and run the program, where "SGX_MODE=SIM" sets the compilation in simulation mode:

```
$ make SGX_MODE=SIM
$ ./secure_rng
```

The output will be similar to the following, although the number will change:

Random Number: 1510565941



If you encounter an error during the program compilation, please refer to the "troubleshooting" section.

4. (Optional Bonus) Develop your own SGX program - Digital Wallet

Submission: Please: (1) save the terminal output to the file "Bonus_DigitalWallet.txt" when you are compiling and running your program; (2) submit your program along with the file "Bonus_DigitalWallet.txt".

In this section, we will develop a digital wallet using SGX. Please implement the digital wallet according to the following specifications:

1. The digital wallet has an initial balance of "1000" securely stored within the enclave.
2. It uses a "transactions" data structure within the enclave to log every deposit and payment. Each entry in this log includes the type of action ("deposit" or "pay") and the amount involved. For example: Deposit: 200; Pay: 50.
3. The digital wallet can support four main functions: **deposit**, **pay**, **view balance**, and **view transactions**. Each function should be implemented as an ECALL:
 - **Deposit:** Increases the current balance by the input amount and logs this deposit in the "transactions" structure.
 - **Pay:** Decreases the current balance by the input amount and logs this payment in the "transactions" structure.
 - **view balance:** Calls an OCALL to display the current balance on the console. You can find the example OCALL code in the "hello world" example.
 - **Check transaction:** Calls an OCALL to display the most recent 3 transactions. If "transactions" contains fewer than three entries, display all of them.
4. In your untrusted application file (App.cpp), please call all four ECALLs to test their functionality.

5. (Optional Bonus) Run the SGX programs in hardware mode

Before installing the full SGX software stack, please verify if your device supports SGX. You can go to the BIOS to see if SGX can be enabled. If you find the SGX option in the BIOS, set it to enabled. For more information about supported devices, please refer to: <https://github.com/ayeks/SGX-hardware>  (<https://github.com/ayeks/SGX-hardware>). In case your device does not support SGX, please contact the instructor for a supported device to work on this bonus task.

The Linux Intel SGX software stack consists of Intel SGX driver, Intel SGX SDK, and Intel SGX Platform Software (PSW). For the full SGX software stack installation, please refer to the Linux SGX Github page:

<https://github.com/intel/linux-sgx>  (<https://github.com/intel/linux-sgx>). After the installation, you can run the program in hardware mode.

5.1 Run the SGX “hello world” example in hardware mode

Submission: Please (1) save the terminal output when you are compiling and running the random number generator program in hardware mode. The output should be saved to the file "Bonus_Hardware_1.txt"; (2) submit the file "Bonus_Hardware_1.txt".

Please go to the folder "Lab1_HelloEnclave" and compile the program in hardware mode:

```
$ make clean  
$ make
```

You will see the following message if the program is successfully compiled in hardware mode:

```
The project has been built in debug hardware mode.  
make[1]: Leaving directory '${sgx-lab-path}/sgx_lab/Lab1_HelloEnclave'
```

Then you can run the SGX program:

```
$ ./app
```

The following message indicates that the program runs successfully:

```
Hello world
```

5.2 Run the SGX random number generator in hardware mode

Submission: Please (1) save the terminal output when you are compiling and running the digital wallet program in hardware mode. The output should be saved to the file "Bonus_Hardware_2.txt"; (2) submit the file "Bonus_Hardware_2.txt".

Please go to the folder "Lab2_WriteEnclave/SampleEnclave" and compile the program in hardware mode:

```
$ make clean  
$ make
```

You will see the following message if the program is successfully compiled in hardware mode:

```
The project has been built in debug hardware mode.  
make[1]: Leaving directory '${sgx-lab-path}/sgx_lab/Lab2_WriteEnclave/SampleEnclave'
```

Then you can run the SGX program:

```
$ ./secure_rng
```



The following message indicates that the program runs successfully, although the number will change:

```
Random Number: 1510565941
```

References

1. Intel® Software Guard Extensions:

<https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/linux-overview.html> (<https://www.intel.com/content/www/us/en/developer/tools/software-guard-extensions/linux-overview.html>)

2. SGX 101: <https://sgx101.gitbook.io/sgx101> (<https://sgx101.gitbook.io/sgx101>)

3. linux-sgx: <https://github.com/intel/linux-sgx> (<https://github.com/intel/linux-sgx>)

4. Hello World for Intel SGX: https://github.com/tatetian/sgx_hello_world (https://github.com/tatetian/sgx_hello_world)

5. Intel SGX Developer Guide: <https://www.intel.com/content/www/us/en/content-details/671334/intel-software-guard-extensions-intel-sgx-developer-guide.html> (<https://www.intel.com/content/www/us/en/content-details/671334/intel-software-guard-extensions-intel-sgx-developer-guide.html>)

Troubleshooting

If you see the following error when compiling the SGX program:

```
make[1]: Entering directory '${sgx-lab-path}/sgx_lab/Lab1>HelloEnclave'

make[1]: *** No rule to make target '/opt/intel/sgxsdk/bin/x64/sgx_edger8r', needed by 'App/Enclave_u.h'. Stop.

make[1]: Leaving directory '${sgx-lab-path}/sgx_lab/Lab1>HelloEnclave'

make: *** [Makefile:182: all] Error 2
```

It indicates that you haven't set the environment variables. Please set the environment variables (you can refer to the SGX SDK installation part), and then compile the SGX program.